Laboratorium 04 - Flutter - Wprowadzenie

Wprowadzenie do Frameworka Flutter

Flutter to framework stworzony przez Google, który umożliwia tworzenie aplikacji mobilnych, webowych i desktopowych z jednej bazy kodu. Flutter bazuje na języku **Dart** i pozwala w prosty sposób budować nowoczesne interfejsy użytkownika.

Kluczowe Cechy Fluttera

- **Wieloplatformowość:** Jedna baza kodu działa na Androidzie, iOS, w przeglądarce i na komputerze.
- **Hot Reload:** Zmiany w kodzie widoczne są niemal natychmiast, bez ponownego uruchamiania aplikacji.
- **Bogaty zestaw widgetów:** Flutter oferuje gotowe komponenty UI (np. przyciski, pola tekstowe, listy).
- **Wydajność:** Aplikacje kompilowane są do natywnego kodu i działają płynnie.

---

1. Utworzenie projektu

W terminalu wpisz:

```
flutter create first_project
cd first_project
```

Następnie możemy otworzyć kod w naszym edytorze kodu.
Na początku możemy usunąć takie foldery jak:

- linux
- macos
- web
- windows
- test

Testami zajmiemy się w przyszłości, a pozostałe foldery nie będą nam potrzebne, bo będziemy tworzyć aplikację tylko na **ios** oraz **android**.

---

2. Minimalna aplikacja w Flutterze

Głównym plikiem, na którym będziemy pracować jest plik `main.dart` w folderze `lib`.

Aby zacząć łatwiej, podmień istniejący kod na poniższy:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

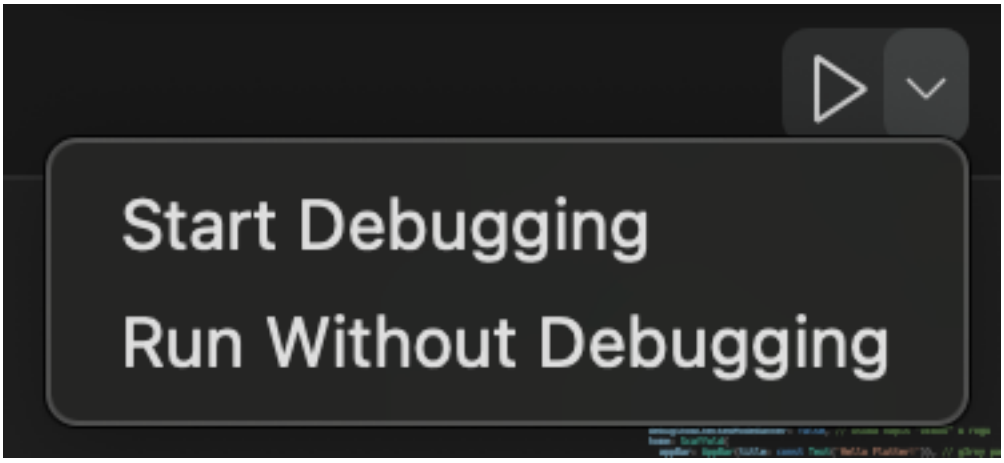
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false, // usuwa napis "DEBUG" w rogu
      home: Scaffold(
        appBar: AppBar(title: const Text('Hello Flutter!')), // górny pasek
        body: const Center( // parent, który ustawia dziecko na środku
          child: Text('Witaj w Flutterze 🎉'), // child - zwykły tekst
        ),
      ),
    );
  }
}
```

👉 Wyjaśnienie:

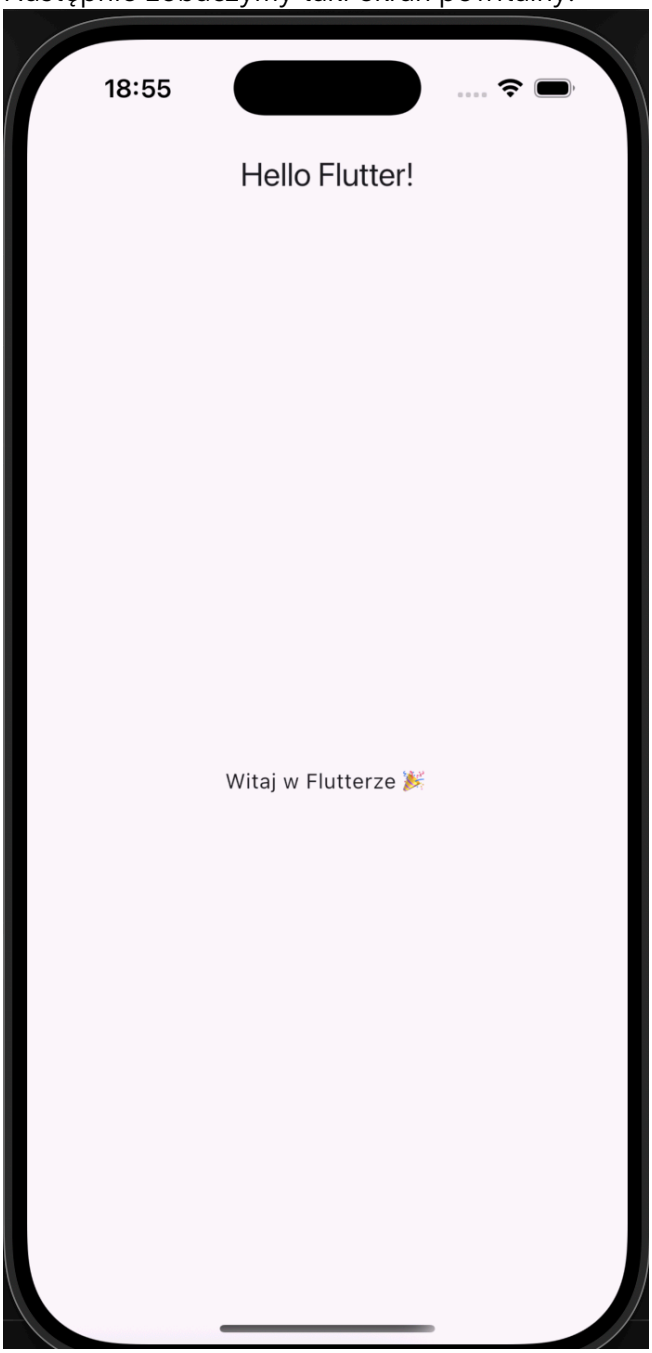
- `runApp()` uruchamia aplikację.
- `MaterialApp` to główny widget aplikacji w stylu Material Design.
- `Scaffold` daje podstawowy układ: pasek u góry (AppBar), tło i obszar roboczy.
- `Center` ustawia zawartość na środku ekranu.
- `Text` to widget, który wyświetla napis.

Tutaj `Center` **to parent**, a `Text` **to child**.

Teraz można uruchomić aplikację np. poprzez `Run Without Debugging`



Następnie zobaczymy taki ekran powitalny:



3. Dodanie przycisku

Podmień widget `Text` na poniższy kod, dodatkowo usuń `const` z **Center**

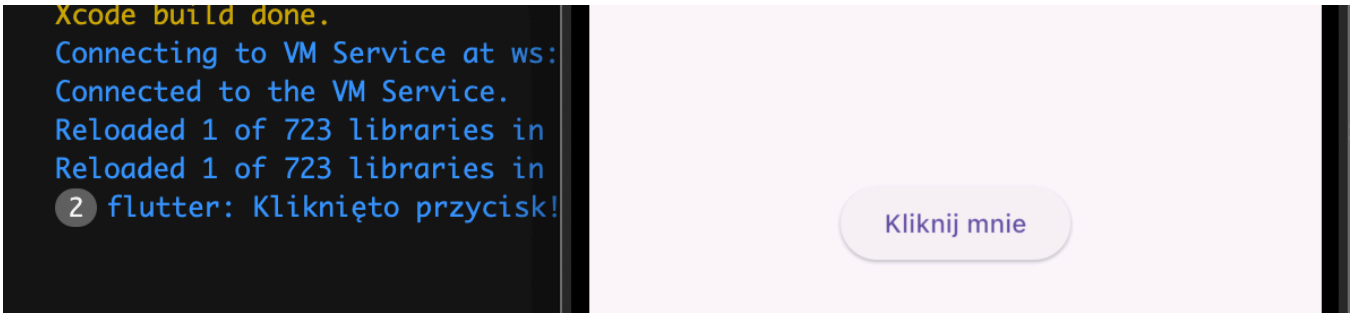
```
ElevatedButton(
  onPressed: () {
    print('Kliknięto przycisk!');
  },
  child: const Text('Kliknij mnie'),
)
```

👉 Wyjaśnienie:

- ElevatedButton to widget przycisku.
- onPressed to funkcja, która wykona się po kliknięciu. Tutaj po prostu wypisujemy coś do konsoli.
- child określa zawartość przycisku (tutaj: tekst).

Czyli **przycisk jest parentem, a jego zawartość (tekst) to child**.

Efekt:



---

4. Kilka widgetów obok siebie – Column

Aby ułożyć elementy jeden pod drugim, używamy **Column**:

```
Column(
  mainAxisAlignment: MainAxisAlignment.center, // wyśrodkowanie w pionie
  children: [
    const Text('To jest pierwszy napis'),
    ElevatedButton(
      onPressed: () {
        print('Przycisk działa!');
```

```
      },
      child: const Text('Kliknij mnie'),
    ),
  ],
)
```

👉 **Wyjaśnienie:**

- Column to widget, który układa swoje elementy (**children**) pionowo.
- Wcześniej używaliśmy tylko jednego elementu (child), ale jeśli chcemy mieć **więcej niż jeden widget**, musimy użyć właściwości children.
- child → **pojedynczy widget**
- children → **lista widgetów** (czyli tablica w nawiasach kwadratowych [], oddzielone przecinkami).
- W powyższym przykładzie children zawiera dwa elementy: Text i ElevatedButton.
- mainAxisAlignment.center ustawia całą zawartość na środku ekranu w pionie.

Finalny kod:

```
@override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false, // usuwa napis "DEBUG" w rogu
      home: Scaffold(
        appBar: AppBar(title: const Text('Hello Flutter!')), // górny pasek
        body: Center(
          child: Column(
            mainAxisAlignment:
                MainAxisAlignment.center, // wyśrodkowanie w pionie
            children: [
              const Text('To jest pierwszy napis'),
              ElevatedButton(
                onPressed: () {
                  print('Przycisk działa!');
                },
                child: const Text('Kliknij mnie'),
              ),
            ],
          ),
        ),
      ),
    );
  }
```

```
}
```

Zadania Praktyczne 💻

1. **Pierwsza aplikacja**

   Utwórz nowy projekt Flutter i uruchom go na emulatorze lub telefonie.
2. **Zmiana tekstu**

   Zmień domyślny napis w aplikacji na swoje imię.
3. **Dodanie przycisku**

   Dodaj przycisk ElevatedButton, który po kliknięciu wyświetli w konsoli "Przycisk został kliknięty!".
4. **Pozycjonowanie**

   Użyj `Column`, aby wyświetlić dwa napisy i przycisk jeden pod drugim.

Przydatne Linki 🔗

- **Flutter na DartPad:** https://dartpad.dev/flutter
- **Oficjalna Dokumentacja Fluttera:** https://docs.flutter.dev/
- **Widget Catalog:** https://docs.flutter.dev/ui/widgets